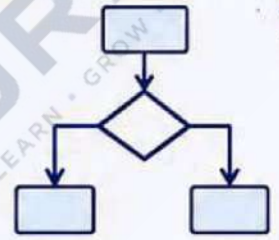
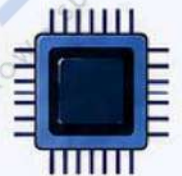


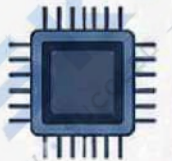
1010101010
0101010101
1010101010
0101010101



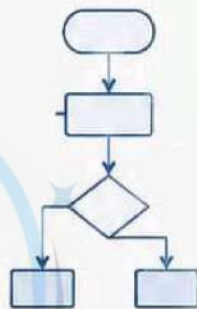
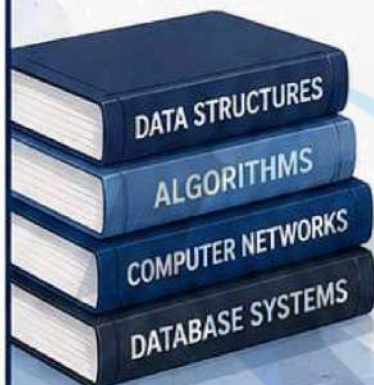
Notes



Unit-I



10110
01001
10101



U1 -

Date / /

Page No.

* Explain OOP, Features & Advantages.

Introduction-

Object Oriented Programming (OOP) is a programming methodology in which programs are organized using objects and classes instead of only functions and procedures.

In OOP, data and functions are combined together into a single unit called as object.

It helps programmers to develop programs that are secure, reusable and easy to manage.

OOP is mainly used in programming languages like -

- C++
- Java
- Python
- C#
- Ruby

OOPs is widely used for developing

- Software Applications
- Mobile Applications
- Games
- Web applications
- Banking Systems

Defination of OOP.

Object Oriented Programming is a programming approach in which programs are designed using objects, classes and data handling concepts such as encapsulation, inheritance, abstraction and polymorphism.

Features of OOP-

The following are the important features of OOP-

1. Class-

A class is the blueprint or a template which is used to create object.

It contains-

- Data members (Variables)
- Member Functions (Functions).

Eg-

```
class Student
```

```
{
```

```
    int roll;
```

```
    void display();
```

```
};
```

} Student Class.

2. Object-

An object is an instance of class. Objects are used to access the members of class.

Eg- Student s1;

Here, s1 is an object of class Student.

3. Encapsulation -

Encapsulation means binding data and functions together into a single unit.

It also protects data from unauthorized access.

Eg-

Class combines -

- Variables
- Functions

into one unit.

Advantages -

- Data Security.
- Better control over data.

4. Abstraction -

Abstraction means hiding unnecessary details and showing only important information to the user.

Eg- When driving a car, we only use -

- Steering
- Break
- Accelerator

we do not know the internal engine mechanism

Adv - Reduces complexity
Impr security

5. Inheritance-

Inheritance is the process in which one class acquires the properties of another class

The existing class is called-

- Base class / Parent class

The new class is called:

- Derived class / child class.

Advantage-

- Code reuseability
- Reduce duplication.

6. Polymorphism-

Polymorphism means "many forms!"

The same function or operator behaves differently in different situations.

Types-

1. Compile Time
2. Run Time.

Eg-

Function Overloading.

7. Dynamic Binding-

In dynamic binding, the function call is connected to the function body at runtime.

It is mainly used with virtual function.

8. Message Passing-

Objects communicate with each other by sending messages.

Eg-

One object calls the function of another object.

Advantages of OOP-

The following are the advantages-

1. Code Reuseability-

In OOP, the code is reusable. Using inheritance, existing code can be reused many times.

This reduces program size and development time.

2. Data Security-

Encapsulation and data hiding protect data from unauthorized access.

3. Easy Maintenance-

Programs using OOPs are easier to modify and manage.

4. Modularity-

Programs are divided into small independent objects.

This made debugging easier.

5. Flexibility -

6. Easy to Upgrade

7. Reduces Complexity

8. Better realworld Modeling

- 1) Real time System
- 2) Simulation & Modeling
- 3) Computer Graphics & Multimedia
- 4) Game development
- 5) Web development
- 6) Mobile Application
- 7) AI & Expert System
- 8) CAM/CAD System.

Applications of OOP -

OOP is used in many areas of computer science and software development.

The important applications of OOP are explained below.

1) Real Time System.

OOP is widely used in real time systems where quick response and accuracy are required.

Eg-

- Railway reservation system
- Air traffic control system
- Banking System

Advantages -

- It provides better data handling
- Faster Program execution
- Easy maintenance.

2. Simulation & Modeling.

OOP is used to create real world simulations and models.

Objects represents real life entities such as-

- Cars
- Machines
- Birds
- Animals
- Humans

Eg-

- Weather forecasting System
- Scientific Simulations

Advantages-

- Realistic Modeling
- Reusable Codes
- Easy Modification

Date / /
Page No.

3. Computer Graphics & Multimedia -

OOP is heavily used in graphics applications because graphics program required many objects such as -

- Images
- Shapes
- Animations

Eg. - Animation Software

- Image editing software
- Video editing application

Advantages -

- Better Organization of graphics object
- Easy handling of complex designs

4. Game Development -

Game applications used OOP concepts extensively.

In games -

- Enemies
 - Players
 - Weapons
 - Vehicles
- are treated as objects.

Eg. - Free Fire • PUBG • Minecraft

- Racing Games

Advantages -

- Easy Game design
- Reusability of code
- Better Game Control

5) Web Development-

Modern web applications use OOP for creating dynamic and interactive websites.

Eg. E-Commerce website

- Social Media platforms
- Online learning system

Advantages-

- Secure applications
- Better database connectivity
- Easy management of large projects

6. Mobile Application Development

Mobile applications are mainly developed using OOP languages such as-

- Java | Kotlin | Swift.

Eg - Android apps
iOS apps

Advantages-

- Reuseability of Code
- Better user interface management
- Easy updating

7. Artificial Intelligence and Expert Syst

OOP is used in Artificial Intelligence (AI) applications because AI systems required complex data handling.

Eg. • Chatbots • Robotics
Machine Learning

Advantages-

- Flexible programming
- Easy Problem Solving
- Better data Organization.

8. CAD / CAM System

CAD means Computer Aided Design and CAM - Computer Aided Manufacturing

OOP is used in designing.

- Machines
- Buildings
- Electronic circuits.

Advantages-

- Accurate design
- Easy Modification
- Time Saving

* Structured Vs OOP (Face of)

Introduction-

Structured programming and object oriented programming (OOP) are two important programming approaches used in software development.

Structured Programming focuses mainly on-

- Functions
- Procedures
- Sequential execution.

While object oriented programming focuses on-

- Objects
- Classes.
- Data Security
- Reuseability

Structured programming is suitable for small programs, while oop is better for large and complex software applications.

Difference between

Structured

OOP

- | | |
|--|--|
| 1) SP focuses on functions and procedures | 1) OOP focuses on objects and classes |
| 2) In SP programs are divided into functions | 2) In OOP programs are divided into objects. |
| 3) It follows top-down approach | 3) It follows bottom up approach |
| 4) In SP data and functions are separate | 4) In OOP data and functions are combined |
| 5) It is less secure because data is globally accessed | 5) It is more secure due to data hiding |
| 6) In SP reuseability is limited | 6) In OOP, higher code reuseability through inheritance. |
| 7) Difficult to manage large programs | 7) Easy to manage large and complex programs |

8) SP does not support inheritance and polymorphism

9) Suitable for small application

10) The modification of SP is difficult

11) Less real-world modeling

Eg- C, Pascal

8) OOP supports inheritance and polymorphism

9) Suitable for large applications

10) The modification of OOP is easy

11) Better real-world representation

Eg- C++, Java, Python

* Data Abstraction

Introduction-

Data abstraction is one of the most important features of OOP.

Abstraction means hiding unnecessary implementation details and shows only important information to the user.

In simple words the user only know -

- What an object does

but does not know -

- How it works internally.

Abstraction helps in reducing complexity and improving security.

Defination-

Data Abstraction is the process of hiding internal implementation details and showing only important functionalities to the user.

Real-Life Example

A common real-life example is an ATM Machine.

When we use an ATM -

- We insert the card
- Enter the PIN
- Withdraw money

But we do not know -

- How the machine processes transaction internally.
- How the bank server works.

Thus, unnecessary details are hidden from user.

Need of Abstraction -

Data abstraction is needed because -

- It reduce program complexity
- It improve security
- It hides sensitive data

Syntax -

```
class ClassName  
{  
    private:  
        data_members;  
  
    public:  
        member_functions;  
}
```

Example program -

```
#include <iostream>  
using namespace std;  
  
class Student  
{  
    private:  
        int marks;  
    public:  
        void setMarks (int m)  
        {  
            marks = m;  
        }  
        void display ()
```

Advantages-

1. Improve Security
2. Reduce Complexity
3. Easy Maintenance
4. Increase Reuseability

Disadvantages-

1. Increase Program Complexity
2. More planning required

Application-

- Banking System
- ATM Software
- Mobile Applications

* Encapsulation-

Introduction-

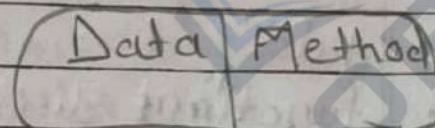
Encapsulation is one of the fundamental features of OOP.

The word "Encapsulation" means:

wrapping or binding data & function together into a single unit.

In C++, encapsulation is achieved using class.

It helps in:



- data Security
 - data hiding
 - controlled access to data
- class

Definition-

Encapsulation is the process of combining data members and member functions into a single unit called as class and restricting direct access to data.

Real Life example -

Example: Capsule Medicine -

A medicine capsule contains -

- Medicine powder
- Protective covering

combined into one unit -

Similarly, in OOP -

- data
 - functions
- are combined together inside a class.

Encapsulation works in C++.

Encapsulation is implemented using -

- Class
- Access Specifier

Access Specifiers -

- Private
- Public
- Protected

Usually -

data members - Private
Function - public

Syntax -

```
class Class_Name  
{  
    private:  
        // data members  
    public:  
        // member functions  
};
```

Example- For Abstraction & Encapsulation

```
#include <iostream>
using namespace std;

class Student
{
    private:
        int marks; // hidden data

    public:
        void setMarks (int m)
        {
            marks = m;
        }

        void display()
        {
            cout << "Marks = " << marks;
        }
};

int main ()
{
    Student s1;
    s1.setMarks(90);
    s1.display ();

    return 0;
}
```

Advantages

1. Data Security
2. Data Hiding
3. Better Control
4. Easy Maintenance
5. Improve Code Reuseability

Disadvantages-

1. More Complex design
2. Slightly Increased Program Size
- 3.

Application-

- Banking System
- ATM Software
- Employees management System
- Online Applications.

* Class and Object

Class-

A class is a user-defined data type that contains -

- data members (variables)
- member functions (functions)

A class is a blue print or template used to create objects.

It does not occupy memory untill an object is created.

Defination-

A class is a collection of data member and member functions. combined together into a single unit.

Syntax-

```
class ClassName
```

```
{
```

```
    private:
```

```
        // data members
```

```
    public:
```

```
        // member-function
```

```
};
```

Eg-

```
class Student
```

```
{
```

```
    public:
```

```
        int roll;
```

```
        void disp();
```

```
{
```

```
    cout << "Student  
    Dada"; }
```

```
};
```

* Object -

An object is an instance of class

Objects are used to access the members of a class.

When an object is created:

- Memory is allocated
- class members can be used.

Defination -

An object is a real entity created from a class that contains actual data and can perform operations.

Syntax -

```
className objectName;
```

Eg -

```
Student st;
```

Here,

st is the object of class Student

Real life Example

Example: Car

Class:

A class act like a blueprint of a car.

It contains common properties such as:

- Color / Model / Speed

and functions such as

- Start() | Stop()

Objects:

- Actual car created using the blueprint are objects.

Eg-

BMW cars

Audi cars

Tesla cars

- All cars have different colors
different speeds

But they are created from the
same blueprint

* Defining a Class

```
#include <iostream>
using namespace std;
```

```
// Function Prototype
int add (int, int);
```

```
int main()
```

```
{
```

```
    int a=10, b=20, sum;
```

```
    sum = add(a, b);
```

```
    cout << "Sum = " << sum;
```

```
    return 0;
```

```
}
```

```
// Function Definition
```

```
int add (int x, int y)
```

```
{
```

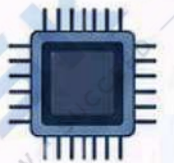
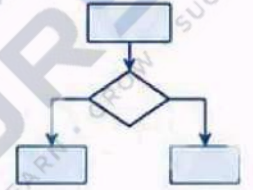
```
    return x+y;
```

```
}
```

Output -

Sum = 30.

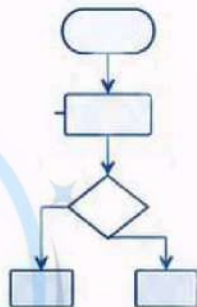
1010101010
0101010101
1010101010
0101010101



10110
01001
10101



Unit - II



U-2

Date / /
Page No.

* Function Prototype.

Introduction-

In C++, a function is used to perform a specific task.

Before a function is used, the compiler should know about the functions.

- Name
- Return type
- number of arguments
- data types of arguments

This information is provided through a function prototype.

A function prototype tells the compiler that a function exists and may be defined later in the program.

Definition of function prototype

A function prototype is a declaration of a function that specifies the function name, return type and parameters before its actual definition.

Eg- `int add (int, int)`

Syntax - `return-type
function-name (parameters-list)`

* Inline Function-

Introduction-

In C++, functions are used to perform specific tasks.

Whenever a function is called, the program transfers control to the function and returns back to execution. This process takes some extra time.

For small functions that are called frequently, C++ provides the Inline Function feature to improve execution speed.

An Inline function replaces the function call with the actual function code during compilation.

Definition-

An Inline function is a function in which the compiler replaces the function call with the actual function code during compilation.

The keyword `inline` makes a function inline function.

Syntax -

```
inline return_type  
function_name (parameters)  
{  
    // function body  
}
```

Eg -

```
inline int square (int x)  
{  
    return x * x;  
}
```

Eg of Program -

```
#include <iostream>  
using namespace std;
```

```
inline int square (int x)  
{  
    return x * x;  
}
```

```
int main ()
```

```
{    cout << "Square = " << square(5)
```

```
    return 0;
```

```
}
```

Output = 25

* Function Overloading.

Introduction-

Function Overloading is an important feature of OOP in C++.

It allows multiple functions to have the same name but different parameters. Lists.

The compiler distinguished overloaded functions based on-

- Number of arguments
- Types of arguments
- Order of arguments

Function Overloading is an example of Compile time Polymorphism.
(Static Polymorphism).

Defination

Function Overloading is a feature in C++ that allows two or more functions to have the same name but different parameters within the same class or program.

Function overloading is used to -

- perform similar operations using the same function name.
- Improve code reusability
- Reduce the number of function names
- Archive compile-time polymorphism.

Rules of Overloading -

Functions can be overloaded by changing;

1. Number of Parameters.

Eg- `int add(int a, int b);`
`int add(int a, int b, int c);`

2. Data Types of Parameters.

Eg- `int add(int a, int b);`
`float add(float a, float b);`

3. Order of Parameters.

Eg- `void display(int, float);`
`void display(float, int);`

Syntax -

```
return_type  
function_name (parameter_list 1);
```

```
return_type  
function_name (parameter_list 2);
```

```
return_type  
function_name (parameter_list 3);
```

Example - By num of arguments

```
#include <iostream>  
using namespace std;
```

```
int add (int a, int b)  
{  
    return a+b; }
```

```
int add (int a, int b, int c)  
{  
    return a+b+c; }
```

```
int main()  
{  
    cout << sum = "add (10, 20) << endl;  
    cout << sum = " << add (10, 20, 30) << endl;  
    return 0;  
}
```

* Constructor -

Introduction -

A constructor is a special member function of a class that is automatically called when an object is created.

The main purpose of constructor is to initialize the data members of an object.

Constructor plays an important role in OOP because they help in initialize objects at the time of creation.

Definition of Constructor

A constructor is a special member function of a class that has same name as the class name and is automatically called when an object is created.

Characteristics of Constructor

- 1) Constructor name must be the same as the class name.
- 2) It has no return type, not even void.
- 3) It is called automatically when an object is created.
- 4) It is used to initialize data member.
- 5) A constructor can be overloaded.
- 6) Constructors can be public, private, protected.

Syntax-

```
class ClassName
{
    public:
        ClassName()
        {
            // initialization code
        }
};
```

Eg-

```
#include <iostream>
using namespace std;
```

```
class Student
{
public:
    Student()
    {
        cout << "Const called";
    }
};
```

```
int main()
{
    Student s1;
    return 0;
}
```

Constructors are used to-

- Initialize object data
- Reduce coding effort
- Improve reusability
- Automatically perform setup operations.

Types of Constructor

The three main types of constructors are -

1) Default Constructor

2) Parameterized Constructor

3) Copy Constructor.

1) Default Constructor -

A type of constructor that does not accept any arguments is called as default constructor.

It initializes the object with default values.

Syntax -

```
class ClassName
{
    public:
        ClassName()
        {
            // constructor body
        }
};
```

Example Program -

```
#include <iostream>
using namespace std;
```

```
class Student
```

```
{
```

```
public:
```

```
    Student()
```

```
{
```

```
    cout << "Default Const called";
```

```
}
```

```
};
```

```
int main()
```

```
{
```

```
    Student s1;
```

```
    return 0;
```

```
}
```

Output- Default Const called

When an object s1 is created

```
Student s1;
```

The default constructor is automatically called because no argument is passed.

Advantages of Default Constructor.

- Automatic object initialization
- Easy to use
- Reduces coding efforts

2. Parameterized Constructor.

A type of constructor that accepts one or more arguments is called as parameterized constructor.

It allows different objects to be initialized with different values.

Syntax-

```
class ClassName  
{  
    public:  
        ClassName (Parameter list)  
        {  
            // initialization code  
        }  
};
```

Eg-

```
#include <iostream>
using namespace std;

class Student
{
public:
    int roll;
    Student (int r)
    {
        roll = r;
    }
    void display()
    {
        cout << "Roll Num = " << roll;
    }
};

int main()
{
    Student s1(100);
    s1.display();
    return 0;
}
```

Output - Roll Num = 100

Explanation-

The constructor receives values 100 and stores it in the variable roll.

```
Student s1(100);
```

Thus an object is initialized with a user defined values.

Advantages of Parameterized

- Initializes object with different values.
- More flexible than default constructor.
- Reduces the need for separate initialization function.

3. Copy Constructor:

A type of constructor that initialized one object using another object of the same class is called as Copy Constructor.

It creates a copy of an existing object.

Syntax-

```
Class Name (Classname &object)
{
    // Copy Code
}
```

Example -

```
#include <iostream>
using namespace std;
```

```
class Student
```

```
{
```

```
    public:
```

```
        int roll;
```

```
    Student (int r)
```

```
    {
```

```
        roll = r;
```

```
    }
```

```
    Student (Student &s)
```

```
    {
```

```
        roll = s.roll;
```

```
    }
```

```
    void disp()
```

```
    {
```

```
        cout << "Roll Num = " << roll;
```

```
    }
```

```
};
```

```
int main()
```

```
{
```

```
    Student s1 (101);
```

```
    Student s2 (s1);
```

```
    s2.disp();
```

```
    return 0;
```

```
}
```

Output -

Roll num = 101

Explanation -

Object s2 is created by copying the data of object s1.

Student s2(s1);

The copy constructor copies the values of roll from s1 to s2.

Advantages -

- Creates duplicate objects easily
- Saves programming efforts
- Useful when passing object to function

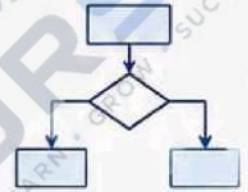
Conclusion -

Constructors are special member function used to initialize object automatically.

There are three types

1. Default
2. Parameterized
3. Copy

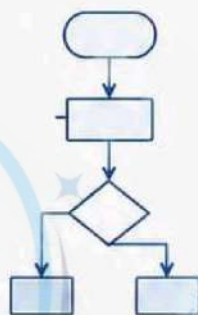
1010101010
0101010101
1010101010
0101010101



10110
01001
10101



Unit - III



U3-

Date / /
Page No.

* Access Specifier.

Introduction-

Access specifiers are an important feature of Object Oriented programming (OOP) in C++.

They are used to control the accessibility of data members and member functions of a class.

Access specifiers determine -

- Which members can be accessed from outside the class.
- Which members can be accessed only within the class.
- Which members can be accessed by derived classes.

Thus, access specifiers help in archiving data handling and encapsulation.

Definition-

Access specifiers are keywords used in a class to define the access level of its data members and member functions.

In C++, there are three types of access specifiers.

1. Public
2. Private
3. Protected

Types of Access Specifiers

1. Public Access Specifier

Members declared as public can be accessed from anywhere in the program.

Characteristics -

- Accessible inside the class
- Accessible through the object
- Accessible outside the class

Syntax -

```
class ClassName
{
    public:
        // data member
        // member functions
};
```

Eg -

```
#include <iostream>
using namespace std;
```

```
class Student
```

```
{
```

```
    public:
```

```
        int roll;
```

```
};
```

```
int main()
```

```
{
```

```
    Student s1;
```

```
    s1.roll = 101;
```

```
    cout << "Roll Num = " << s1.roll;
```

```
    return 0;
```

```
}
```

Output - Roll Num = 101

Since roll is public, it can be accessed directly by using the object s1.

2. Private Access Specifier

Members declared as a private can be accessed only inside the class.

They cannot be accessed directly from outside the class.

Characteristics.

- Accessible inside the class.
- Not accessible outside the class.
- Provides data security.

Syntax -

```
class ClassName  
{  
    private:  
        // Data member  
        // Member Function  
};
```

Explanation -

The variable marks is private, so it cannot be accessed directly.

St. marks = 85 ; // Error.

Instead public function is used to access.

Eg-

```
#include <iostream>
using namespace std;
```

```
class Student
```

```
{
```

```
    private:
```

```
        int marks;
```

```
    public:
```

```
        void setMarks(int m)
```

```
        {
```

```
            marks = m;
```

```
        }
```

```
        void display()
```

```
        {
```

```
            cout << "Marks = " << marks;
```

```
        }
```

```
    };
```

```
    int main()
```

```
    {
```

```
        Student s1;
```

```
        s1.setMarks(85);
```

```
        s1.display();
```

```
        return 0;
```

```
    }
```

Output -

Marks = 85.

3. Protected Access specifier

Members declared as protected can be accessed -

- Inside the class
- By derived classes (inheritance)

But they cannot be accessed directly through objects outside the class.

Syntax -

```
class ClassName
{
    protected:
        // Data Member
};
```

Output - Roll Num = 101

Explanation -

The protected member roll is accessible in derived class.

Eg-

```
#include <iostream>
using namespace std;
```

```
class Student
{
    protected:
        int roll;
};
```

```
class Result : public Student
{
    public:
        void setData()
        {
            roll = 101;
        }
        void display()
        {
            cout << "Roll Num = " << roll;
        }
};
```

```
int main()
{
    Result r;
    r.setData();
    r.display();
    return 0;
}
```

Comparison

Access Specification	Public	Private	Protected
Inside class	Yes	Yes	Yes
Outside	Yes	No	No
Inherited	Yes	No	Yes

Advantages -

1. Improve data security
2. Support data hiding
3. Helps achieve encapsulation

* Memory Allocation for Object.

Introduction-

In OOP, a class is a blueprint for creating objects.

A class itself does not occupy memory.

Memory is allocated only when objects of the class are created.

Memory allocation for objects is an important concept because every object needs memory to store its data members.

Definition

Memory allocation for objects is the process of reserving memory for the data members of a class when an object is created.

In simple words, memory is allocated only to objects and not to the class itself.

How Memory Allocation Works?

When a class is declared:

```
class Student  
{  
    int roll;  
    float marks;  
};
```

No memory is allocated at this stage.

Memory is allocated only when an object is created.

Student st;

Now, memory is reserved for:

- roll
 - marks
- of object st.

Important points -

1. Class Does not occupy Memory.
→ A class is only a blueprint.

Eg-

```
class Student  
{  
    int roll;  
    float marks;  
}
```

Here, no memory is allocated.

2. Object occupies Memory.

When an object is created:

Student s1;
memory is allocated to all data members.

3. Seperate Memory for Each Object

Each object get its own copy of data members.

Eg- Student s1; / Student s2;

Both objects has seperate memory location.

Program:-

```
#include <iostream>
using namespace std;
```

```
class Student
```

```
{
```

```
public:
```

```
    int roll;
```

```
    float marks;
```

```
};
```

```
int main()
```

```
{
```

```
    Student s1, s2;
```

```
    s1.roll = 101;
```

```
    s1.marks = 99.9;
```

```
    s2.roll = 102;
```

```
    s2.marks = 88.8
```

```
    cout << "Student 1 Roll = " << s1.roll << endl;
```

```
    cout << "Student 2 Roll = " << s2.roll << endl;
```

```
    return 0;
```

```
}
```

* Object as Function Arguments

Introduction-

In C++, objects of a class can be passed to functions just like ordinary variables.

Passing objects to function allows one object to interact with another and helps in performing operations on object data.

When an object is passed as an argument to a function, the function can access the data members and member functions of that object.

The concept is widely used in OOP for code reusability and modularity.

Defination-

Passing object as function arguments means sending an object of a class to a function so that the function can use the data and the methods of that object.

Syntax -

```
class ClassName
```

```
{
```

```
    // data members
```

```
};
```

```
void functionName (ClassName obj)
```

```
{
```

```
    // function body
```

```
}
```

Eg -

```
#include <iostream>
```

```
using namespace std;
```

```
class Student
```

```
{ public:
```

```
    int roll;
```

```
    void getData (int r)
```

```
{
```

```
        roll = r;
```

```
}
```

```
};
```

```
void disp (Student s)
```

```
{ cout << "Roll Num = " << s.roll; }
```

```
int main()
```

```
{ Student s1;
```

```
s1.getData (101);
```

```
disp (s1);
```

```
return 0; }
```

Passing Object by Value-

When an object is passed normally a copy of the object is created.

Syntax -

```
void disp (Student s)
{
    // copy of object
}
```

Passing Object by Reference

To avoid copying the object, we can pass it by reference.

Syntax -

```
void disp (Student &s)
{
    // reference the original object
}
```

Inheritance-

Introduction-

Inheritance is one of the most important features of OOP. It allows a new class to acquire the properties and behavior of an existing class.

Inheritance promotes code reusability, reduces duplication of code, and makes programs easier to develop and maintain.

It establishes a relationship between classes where one class inherits the characteristics of another class.

Definition of Inheritance

Inheritance is the process by which one class acquires the data members and member functions of another class.

Terminologies Used in Inheritance

1. Base Class (Parent Class)

The class whose properties are inherited by another class is called the Base class or Parent Class.

2. Derived Class (Child Class)

The class that inherits the properties of the base class is called as Derived class or Child class.

Syntax of Inheritance

```
class BaseClass
```

```
{  
    // Members
```

```
};
```

```
class DerivedClass: Access-Specifier
```

```
{
```

```
    // Additional Members
```

```
};
```

```
BaseClass
```

Eg-

```
#include <iostream>
using namespace std;
```

```
class Animal
```

```
{
    public:
```

```
    void eat()
```

```
{
```

```
    cout << "Animal is Eating" << endl;
```

```
}
```

```
};
```

```
class Dog : public Animal
```

```
{
```

```
    public:
```

```
    void bark()
```

```
{
```

```
    cout << "Dog is Barking" << endl;
```

```
}
```

```
};
```

```
int main()
```

```
{
```

```
    Dog d;
```

```
    d.eat();
```

```
    d.bark();
```

```
    return 0;
```

```
}
```

Advantages of Inheritance

1. Code Reuseability
2. Reduce Code Duplication
3. Easy Maintenance
4. Support Extensibility
5. Saves Development Time
6. Support Polymorphism
7. Better Real-World Representation

Depending on relationship between classes, inheritance is classified into different types.

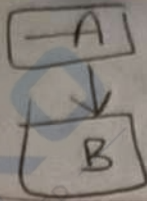
Types of Inheritance

There are five types of Inheritance in C++.

1. Single
2. Multiple
3. Multilevel
4. Hierarchical
5. Hybrid

Application

- Student Management System
- Banking Application
- Hospital Management System



1. Single Inheritance

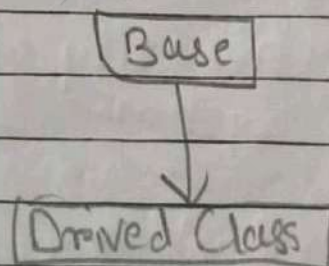
Single Inheritance is the simplest and most commonly used type of inheritance in C++.

In Single Inheritance, one derived class inherits the properties and methods of only one base class.

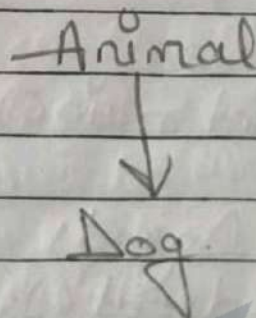
Defination-

Single Inheritance is a type of inheritance in which one derived class inherits the properties and functions of a single base class.

Diagram



Eg



Syntax

```
class BaseClass
{
    // members.
};

class DerivedClass : public BaseClass
{
    // additional members.
};
```

Ex 9

```
#include <iostream>
using namespace std;
```

```
class Animal
{
public:
    void eat()
    {
        cout << "Animal is Eating" << endl;
    }
};
```

class Dog :

```
public Animal
{
```

```
public:
```

```
    void bark()
```

```
{
```

```
    cout << "Dog is Barking" << endl;
```

```
}
```

```
};
```

```
int main()
```

```
{
```

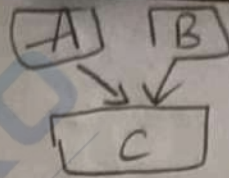
```
    Dog d;
```

```
    d.eat();
```

```
    d.bark();
```

```
    return 0;
```

```
}
```



2. Multiple Inheritance

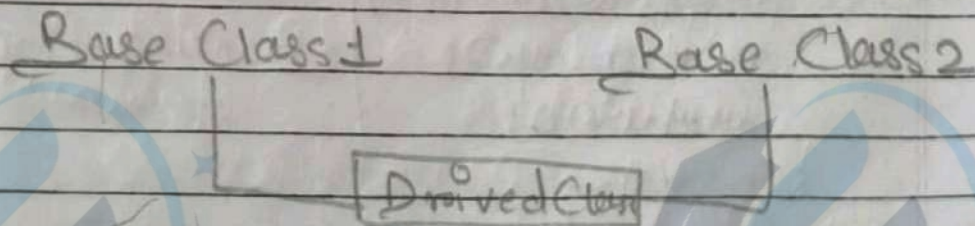
Multiple Inheritance is a type of inheritance in which a single derived class inherits the properties and functions of more than one base class.

It enables a class to combine the features of multiple classes into one class.

Defination-

Multiple Inheritance is a type of inheritance in which one derived class inherits the properties and member functions of two or more base classes.

Diagram-



Representation

Teacher

Student

Result

Syntax -

```
class Base 1  
{  
    //members  
};
```

```
class Base 2  
{  
    // members  
};
```

```
class Derived : public Base 1, public Base 2  
{  
    // members  
};
```

```
int main()  
{  
    Result r;  
    r.getStudent();  
    r.getTeacher();  
    r.display();  
    return 0;  
}
```

Applications -

- University Management System.
- Banking Application

Output -

Student Information
Teacher Information
Result Declared

Program-

#include <iostream>

using namespace std;

class Student

{

public:

void getStudent()

{

cout << "Student Information" << endl;

}

};

class Teacher

{

public:

void getTeacher()

{

cout << "Teacher Information" << endl;

}

};

class Result : public Student, public Teacher

{

public:

void display()

{

cout << "Result Declared" << endl;

}

};



3. Multilevel Inheritance-

Multilevel Inheritance is a type of inheritance in which a class is derived from another derived class.

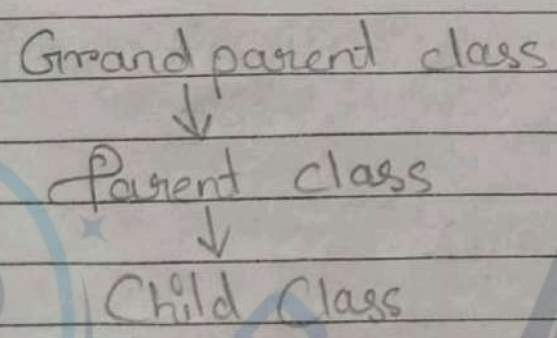
In this Inheritance, there is a chain of inheritance involving more than one level.

It helps in reusing code and creating a hierarchical relationship among classes.

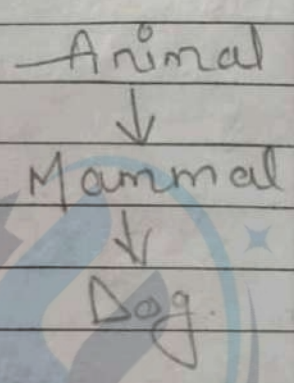
Definition-

Multilevel Inheritance is the type of inheritance in which a derived class becomes a base class for another derived class.

Diagram.



Eg-



Syntax -

```
class Base class  
{  
    // members  
};
```

```
class Derived class1: access Base class  
{  
    // members  
};
```

```
class Derived class2: access Derived  
                        specifier class  
{  
    // members  
};
```

```
#include <iostream>
using namespace std;
```

```
class Animal
```

```
{
    public:
        void eat()
        {
            cout << "Animal is Eating" << endl;
        }
};
```

```
class Mammal : public Animal
```

```
{
    public:
        void walk()
        {
            cout << "Mammal is walking" << endl;
        }
};
```

```
class Dog : public Mammal
```

```
{
    public:
        void bark()
        {
            cout << "Dog is barking" << endl;
        }
};
```

Date / /
Page No.

```
int main()
```

```
{  
    Dog d;
```

```
    d.eat();  
    d.walk();  
    d.bark();
```

```
    return 0;  
}
```

Output:-

Animal is eating
Mammal is walking
Dog is barking.

Applications -

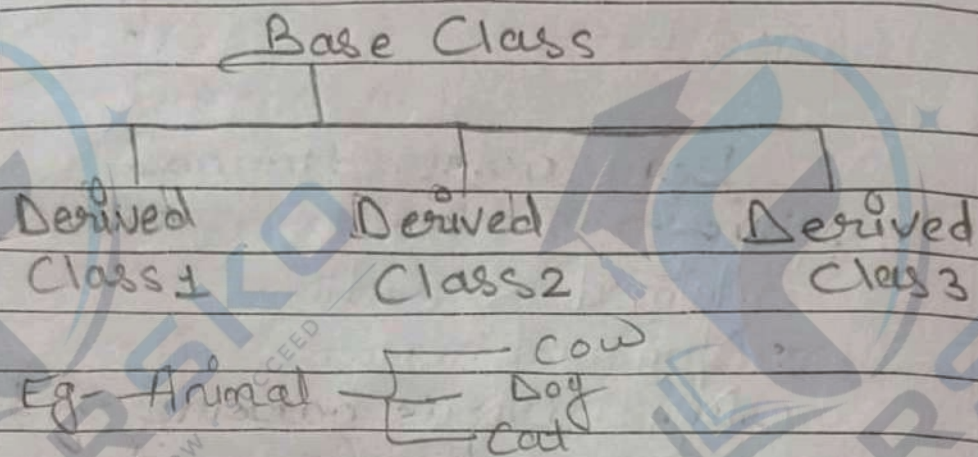
Employee Management System

Banking Application

Hospital Management System

4. Hierarchical Inheritance.

Hierarchical Inheritance is a type of inheritance in which multiple derived classes inherits the properties and functions of one common base class.



Syntax -

```

class Base
{
    // member
};

class Derived 1: public Base
{
    // member
};

class Derived 2: public Base
{
    // member
};

class Derived 3: public Base
{
    // member
};
    
```

Eg-

Date / /
Page No.

```
#include <iostream>
using namespace std;
```

```
class Animal
```

```
{
    public:
        void eat()
        {
            cout << "Animal is Eating" << endl;
        }
};
```

```
class Dog : public Animal
```

```
{
    public:
        void bark()
        {
            cout << "Dog is barking" << endl;
        }
};
```

```
class Cat : public Animal
```

```
{
    public:
        void meow()
        {
            cout << "Cat is Meowing" << endl;
        }
};
```

```
int main()
{
    Dog d;
    Cat c;
    d.eat();
    d.bark();
```

```
c.eat();
c.meow();
return 0;
}
```

Memory Representation

Animal Class

eat()

Dog Class

Eat()

bark()

Cat Class

eat()

meow()

Application -

- School Management System
- Banking System
- Employee Management System

Hybrid Inheritance

Hybrid Inheritance is a type of inheritance which is a combination of two or more types of inheritance. It combines the features of -

- Multiple
- Multilevel
- Hierarchical

to form a more complex inheritance structure.

Hybrid Inheritance is used in large applications.

Application

- Banking Application
- Employee Management System

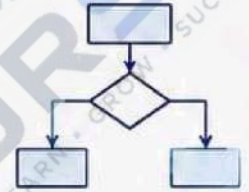
* Visibility Modes.

Visibility Modes are the access specifiers used during inheritance that control the visibility and accessibility of base class members in the derived class.

Syntax -

```
class Derived Class : visib_mode
                        Base Class
{
    // members
}
```

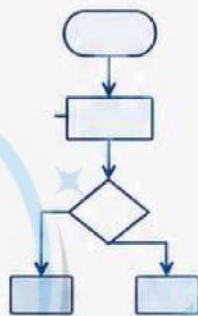
1010101010
0101010101
1010101010
0101010101



10110
01001
10101



Unit - IV



U4-

Polymorphism

Compile

Run

Date / /

Page No.

over operator

* Polymorphism -

Polymorphism is one of the most important features of OOP.

The word polymorphism is derived from two greek words:

- Poly = Many
- Morph = Forms

Thus, Polymorphism means "one interface many forms".

It allows the same function, operator or object to behave differently in different situations.

Polymorphism increases flexibility, reusability, and extensibility of program.

Definition of Polymorphism

Polymorphism is the ability of a function, operator, or object to perform a different task depending on the context in which it is used.

Eg-

Consider a function named:

`display();`

The same function can be used to display -

- Integer values
- Float values
- Character values

with different implementations.

This is called polymorphism.

Types of Polymorphism

Polymorphism in C++ are mainly divided into two types -

- Compile Time Polymorphism
- Run Time Polymorphism

1. Compile Time Polymorphism

Compile Time Polymorphism is a type of polymorphism where the function call is resolved during compilation of the program.

It is also known as -

- Static Polymorphism
- Early Binding
- Static Binding

In Compile Time Polymorphism, the compiler determines which function should be executed before the program runs.

ways to achieve Compile Time P.

In C++, Compile Time Polymorphism is achieved through -

1. Function Overloading
2. Operator Overloading

1. Function Overloading.

Function overloading means defining multiple functions with the same name but different parameters lists.

Eg- `#include <iostream>`
`using namespace std;`

```
class Demo
{
public:
    void show()
    {
        cout << "No-Argument" << endl;
    }
    void show(int x)
    {
        cout << "Value = " << x;
    }
};

int main()
{
    Demo d;
    d.show();
    d.show(10);
    return 0;
}
```

Explanation-

The function name shows is the same, but the parameters are different.

The compiler decides which function to call during compilation.

Hence, it is Compile Time Polymorphism.

2. Operator Overloading

Operator Overloading allows operators to perform different operations for user-defined data types.

Eg- `#include <iostream>`
using namespace std;

```
class Number
{
public:
    int x;
```

```
    Number (int a)
```

```
{
```

```
        x = a;
```

```
}
```

```
    Number operator + (Number n)
```

Calculator

int, int
int, float
float, float
int, int, int

Date / /

Page No.

```
{  
    return Number(x + n * x);  
}
```

```
void display() {
```

```
    cout << x;
```

```
}
```

```
};
```

```
int main() {
```

```
{
```

```
    Number n1(10);
```

```
    Number n2(20);
```

```
    Number n3 = n1 + n2;
```

```
    n3.display();
```

```
    return 0;
```

Output = 30

```
}
```

Explanation.

The + operator is overloaded to add two objects of class Number.

The compiler resolves the operation during compilation.

Therefore, it is compile time polymorphism.

2. Run Time Polymorphism

Run Time Polymorphism is a type of polymorphism in which the function call is resolved during the execution of the program rather than during compilation.

It is also known as -

- Dynamic Polymorphism
- Dynamic Binding
- Late Binding

Run Time Polymorphism is achieved

- Virtual Function
- Function Overriding

How Run Time Polymorphism Works?

In Run Time Polymorphism -

1. A base class contain a virtual function
2. A derived class overrides that function
3. A base class pointer points the derived class object
4. The function call is resolved at runtime

Syntax-

```
class Base
```

```
{
```

```
public:
```

```
virtual void show()
```

```
{
```

```
// base class code
```

```
}
```

```
};
```

```
class Derived : public Base
```

```
{
```

```
public:
```

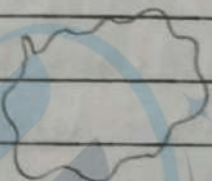
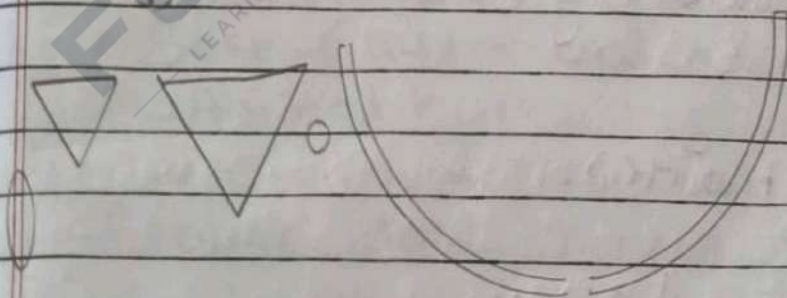
```
void show()
```

```
{
```

```
// derived class code
```

```
}
```

```
};
```



Eg-

```
#include <iostream>
using namespace std;
```

```
class Animal
```

```
{
```

```
public:
```

```
virtual void sound()
```

```
{
```

```
cout << "Animal sound" << endl;
```

```
}
```

```
};
```

```
class Dog: public Animal
```

```
{
```

```
public:
```

```
void sound()
```

```
{
```

```
cout << "Dog Barks" << endl;
```

```
}
```

```
};
```

```
int main()
```

```
{
```

```
Animal *ptr;
```

```
Dog d;
```

```
ptr = &d;
```

```
ptr->sound();
```

```
return 0;
```

```
}
```

Date / /
Page No.

Explanation -

Step 1: Base Class -

```
class Animal  
{ public:  
    virtual void sound();
```

The keyword `virtual` makes the function a virtual function.

Step 2: Derived Class

```
class Dog : public Animal  
{  
    void sound()  
    { cout << "Dog Barks"; }  
}
```

The Dog class overrides the `sound()` function.

Step 3: Base Class Pointer

```
Animal *ptr;
```

A base class pointer is created.

Step 4: Pointer points to Derived Object

```
ptr = &d;
```

The base class pointer points to the derived class object.

Step 5: Function Call

```
ptr->sound();
```

Characteristics

1. Function call is resolved at runtime
2. Archive using virtual function
3. Uses Inheritance
4. Uses functions overriding
5. Also called dynamic polymorphism

Difference Between -

Compile Time Polymorphism	Run Time Polymorphism
1) The function call is resolved during compilation.	The function call is resolved during program execution.
2) It is also known as Static Polymorphism.	It is also known as Dynamic Polymorphism.
3) It uses early binding.	It uses late binding.
4) It is achieved through Function Overloading or Operator Overloading.	It is achieved through Virtual Functions and Function Overriding.
5) Its execution is faster because compiler decides the function before hand.	Its execution is slower because function is decided at run time.
6) Not require inheritance.	Required Inheritance.
Eg - Function Overloading	Eg - Virtual Function

* Virtual Base Class.

Introduction-

In C++, when multiple inheritance and hybrid inheritance are used a problem called Diamond Problem may occur.

This happens when a class inherits the same base class through multiple paths and gets multiple copies of the base class member.

To solve this problem, C++ provides the concept of a Virtual Base Class.

Definition

A virtual base class is a base class that is inherited using the keyword `virtual` so that only one copy of the base class member is inherited regardless of how many times it appears in the inheritance hierarchy.

Characteristics of Virtual base Class

1. Uses the virtual keyword
2. Prevents duplication of base class member
3. Solves the diamond problem
4. Used in Multiple & Hybrid Inheritance
5. Only one copy of base class is

Application -

Virtual base class are used -

- University Management System
- Banking System
- Hospital Management
- Large Enterprise Software

* Virtual Function

A virtual function is an important concept in C++ that is used to achieve Run Time Polymorphism.

Normally, when a base class pointer points to a derived class object the base class function is called.

To call a derived class function using a base class pointer, we use a Virtual Function.

The keyword virtual is used to declare a virtual function.

Definition.

A virtual function is a function that allows a derived class function to be called through a base class pointer during runtime.

Syntax-

```
class BaseClass  
{  
    public:  
        virtual return-type functionname(parameters)  
        {  
            functionbody  
        }  
};
```

Eg-

```
#include <iostream>
using namespace std;

class Base
{
    public:
        virtual void show()
        {
            cout << "Base Class" << endl;
        }
};

class Derived : public Base
{
    public:
        void show()
        {
            cout << "Derived Class" << endl;
        }
};

int main()
{
    Base *ptr;
    Derived d;
    ptr = &d;

    ptr -> show();
    return 0;
}
```

Output - Derived Class

Characteristics

1. Declared using the virtual keyword
2. Defined in the base class
3. Can be overridden in the derived class
4. Used to achieve runtime polymorphism
5. Accessed through a base class pointer or reference
6. Function call is resolved at runtime

Application -

- Graphics Application
- Game Development
- Banking Systems.

* Pure Virtual Function -

Defination -

A pure virtual function is virtual function that has no defination in the class and is initialize with $= 0$.

Syntax -

virtual return-type
function-name() = 0;

```
Eg- #include <iostream>
using namespace std;
class Shape
{
public:
    virtual void area() = 0;
};
class Circle : public Shape
{
public:
    void area()
    {
        cout << "Area of Circle" << endl;
    }
};
int main() { Circle c;
             c.area();
             return 0; }
```

Shub

Date / /
Page No.

Virtual Function	Pure Virtual Function
1) May have function body	Has no function body
2) Derived class may override it.	Derived class must override it.
3) Base class object must be created	Base class object cannot be created.
4) Written as virtual void	Written as virtual void shows = 0;
5) Used for Run Time polymorphism	Used for abstraction and Run Time Polymorphism.

* Early Binding & Late Binding

Binding -

Binding is the process of connecting a function call with the actual function definition that will be executed.

Eg -

show c;

The compiler must decide which show c; function should be executed.

This process is known as binding.

1. Early Binding

Early binding is the process in which the function call is linked with the function body during compilation.

It is also known as -

- Static Binding
- Late Binding

Characteristics of Early Binding

1. Occurs during compilation
2. Use normal functions
3. Faster Execution
4. Does not use virtual function
5. Less flexible.

Eg -

```
#include <iostream>
using namespace std;
class Demo
{
public:
    void show()
    {
        cout << "Hello" << endl;
    }
};
int main()
{
    Demo d;
    d.show();
    return 0;
}
```

Late Binding

Late Binding is the process in which function called is linked with the function body during programming execution.

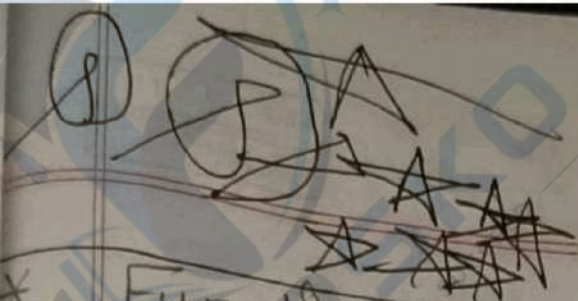
It is also known as -

Dynamic Binding

Late Binding

Eg -

Run Time Polymorphism



* Function Overriding

Function Overriding is an important feature of object oriented programming (OOP) that is used to achieve Run Time Poly.

Defination

Function Overriding is the feature in which a derived class redefines a member function of the base class with the same name and different parameters.

Rules-

1. Inheritance is required
2. Function name must be the same
3. Parameter List must be the same.
4. Return type should be the same.

* Operator Overloading-

Operator Overloading is a feature of C++ that allows programmers to give special meaning to operators when they are used with user-defined data types (Classes & Objects).

In simple words, operator overloading means using the same operator for different purposes according to the type of operands.

Syntax-

```
return_type operator  
    symbol (arguments)  
{  
    // Function body  
}
```